

# Taller 4: Planificación Clásica, Regresiva, Heurística y Jerárquica en Operación Fénix

Inteligencia Artificial – Universidad de los Andes  
Juan Sebastián Maldonado Manrique 202311840  
Santiago Campo González 202325582  
Juan Pablo Reyes Romero 202310852  
Tomás Emilio Velásquez Delgado 202311016

14 de mayo de 2026

## Punto 1: Modelado del dominio de planificación

Para construir el dominio decidimos quedarnos cerca del formalismo STRIPS visto en clase. Representamos los estados como conjuntos inmutables de fluentes y cada acción como una combinación de precondiciones positivas, precondiciones negativas, listas de adición y listas de eliminación. Esta forma de modelar nos ayudó bastante porque cada cambio del mundo podía revisarse con operaciones de conjuntos, sin esconder la lógica del problema detrás de estructuras más complejas.

El dominio usa fluentes como  $At(x, l)$ ,  $Free(l)$ ,  $HandsFree(r)$ ,  $Holding(r, o)$ ,  $Pickable(o)$ ,  $MedicalPost(l)$ ,  $SuppliesReady(l)$  y  $Rescued(p)$ . Con ellos se representa la localización del robot, pacientes y suministros, la disponibilidad de las celdas, la capacidad de carga del robot, la preparación del puesto médico y el cumplimiento final de la misión. La acción *Move* ya estaba dada como referencia y las acciones *PickUp*, *PutDown*, *Rescue* y *SetupSupplies* se completaron manteniendo la semántica esperada por el enunciado.

También dejamos explícitos los objetivos de cada tipo de problema, porque durante las pruebas fue importante no confundir producir una secuencia de acciones con cumplir realmente la meta. En *SimpleRescueProblem*, el objetivo es rescatar al primer paciente del layout, es decir  $goal = \{Rescued(patient\_0)\}$  cuando existe al menos un paciente. En *MultiRescueProblem*, el objetivo es rescatar a todos los pacientes presentes en el layout, por lo que  $goal = \{Rescued(p) \mid p \in patients\}$ . En ambos casos, un estado final solo se considera válido cuando contiene todos los fluentes del objetivo.

| Acción         | Precondiciones                                                                                          | Efectos principales                                                                                                  |
|----------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <i>PickUp</i>  | El robot y el objeto están en la misma celda, el robot tiene las manos libres y el objeto es recogible. | El robot queda sosteniendo el objeto; el objeto deja de estar en la celda y el robot deja de tener las manos libres. |
| <i>PutDown</i> | El robot está en la celda y sostiene el objeto.                                                         | El objeto queda en la celda, el robot recupera las manos libres y deja de sostener el objeto.                        |

| Acción               | Precondiciones                                                                                                 | Efectos principales                                                                                             |
|----------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <i>Rescue</i>        | El robot y el paciente están en el puesto médico, la celda es un puesto médico y los suministros están listos. | El paciente queda marcado como rescatado y deja de estar físicamente en la celda.                               |
| <i>SetupSupplies</i> | El robot está en un puesto médico, sostiene suministros y el puesto todavía no tiene suministros listos.       | El puesto queda con suministros listos, el robot recupera las manos libres y deja de sostener esos suministros. |

Las funciones auxiliares del dominio quedaron implementadas de la manera más directa posible. *is\_applicable* revisa que estén todas las precondiciones positivas y que no aparezca ninguna precondición negativa; *apply\_action* aplica exactamente  $(s - DEL(a)) \cup ADD(a)$ ; y *get\_applicable\_actions* aterriza las acciones con los objetos disponibles antes de filtrar las que sí pueden ejecutarse. Este punto fue clave porque varios errores posteriores no venían de la búsqueda, sino de acciones que parecían razonables pero no respetaban por completo sus condiciones.

## Punto 2: Búsqueda hacia adelante – forwardBFS

La búsqueda hacia adelante fue nuestro punto de referencia porque es la forma más fácil de seguir paso a paso: se parte del estado inicial, se expande por niveles y se detiene cuando aparece un estado que satisface la meta. La frontera se maneja como FIFO, los sucesores se generan con las acciones aplicables y cada nodo guarda el plan que lo produjo. También cambiamos la cola para usar *deque*, porque la versión con listas funcionaba en mapas pequeños, pero hacía operaciones innecesariamente costosas.

Esta búsqueda es completa cuando el espacio de estados alcanzable es finito y, como todas las acciones tienen costo unitario, también es óptima en longitud de plan. Su costo temporal y espacial en el peor caso es  $O(b^d)$ , donde  $b$  es el factor de ramificación efectivo y  $d$  la profundidad de la solución más corta. En la práctica observamos que el consumo crece rápidamente cuando hay más celdas u objetos, porque cada estado combina posición del robot, ubicación de objetos, fuentes de carga y fuentes de rescate.

## Punto 3: Búsqueda regresiva – backwardSearch

La función de regresión se implementó con la regla formal indicada en el taller:

$$REGRESS(g, a) = (g - ADD(a)) \cup PRECOND^+(a).$$

Antes de calcular el objetivo regresado se verifica que la acción sea relevante, es decir, que  $ADD(a) \cap g \neq \emptyset$ , y que no contradiga el objetivo actual, es decir, que  $DEL(a) \cap g = \emptyset$ . Luego *backwardSearch* ejecuta una BFS sobre descripciones parciales de objetivo. Cada nodo de la búsqueda es un conjunto de fuentes que deberían ser verdaderos antes de ejecutar el sufijo de plan ya construido, y la búsqueda termina cuando ese conjunto queda contenido en el estado inicial.

Al implementar regresión vimos que la fórmula por sí sola no era suficiente para que la búsqueda fuera práctica. Si se consideran todas las acciones aterrizadas sin ningún cuidado, aparecen subobjetivos

que en el mundo del rescate no tienen sentido, como una entidad en dos celdas al mismo tiempo o un robot que está libre y sosteniendo un objeto a la vez. Por eso agregamos podas basadas en invariantes del dominio. La idea no fue cambiar la definición de regresión, sino evitar que el algoritmo gastara tiempo explorando metas parciales físicamente imposibles.

La regresión es completa y óptima en longitud cuando la BFS conserva todas las acciones relevantes y todas las podas corresponden a invariantes del dominio. En nuestra solución, las podas usadas se justifican por las reglas físicas del problema: una entidad no puede estar en dos celdas a la vez, el robot no puede sostener dos objetos simultáneamente y los predicados estáticos no pueden aparecer como metas si no existen desde el estado inicial. Los planes retornados se validan además al ejecutarse sobre el simulador.

## Punto 4: Heurísticas para planificación – A\*

El planificador A\* mantiene una cola de prioridad con  $f(n) = g(n) + h(n)$ . El costo  $g(n)$  corresponde a la longitud del plan acumulado y la heurística  $h(n)$  se calcula sobre el estado sucesor. También se guarda el mejor costo conocido para cada estado, lo que evita reexpandir caminos más caros hacia el mismo estado.

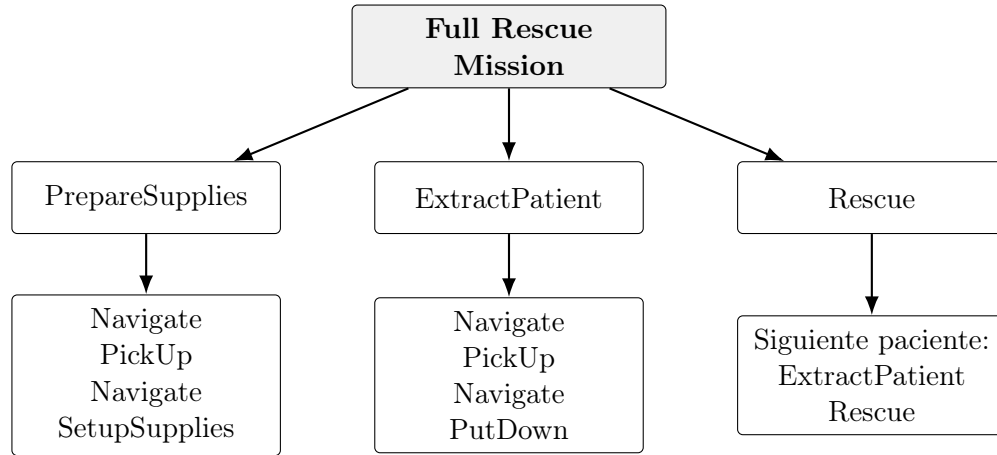
La heurística *ignorePreconditionsHeuristic* se implementó como el conteo directo de fluentes objetivo no satisfechos,  $|g - s|$ . En este dominio, cuando el objetivo contiene fluentes *Rescued*( $p$ ), cada paciente no rescatado exige al menos una acción *Rescue*; por eso este conteo es admisible. En *tinyBase*, por ejemplo, el estado inicial no contiene *Rescued*(*patient\_0*), así que la heurística vale 1, mientras que el plan real más corto tiene 9 acciones. Este ejemplo muestra que no sobreestima el costo. Además, como una acción *Rescue* agrega a lo sumo un fuente de rescate y las demás acciones no reducen este conteo en más de una unidad, la heurística se comporta de manera consistente para los objetivos de rescate evaluados.

La heurística *ignoreDeleteListsHeuristic* resuelve una relajación donde las listas de eliminación se ignoran y el estado solo crece. La versión final aplica una búsqueda local codiciosa en ese espacio relajado: en cada paso elige una acción aplicable que agrega nuevos fluentes, priorizando progreso directo sobre el objetivo y luego crecimiento informativo del estado relajado. Esta heurística fue mucho más informativa en número de estados expandidos, aunque su cálculo es más costoso porque simula varios pasos relajados para cada estado consultado. Como usa hill-climbing y no garantiza encontrar el plan relajado óptimo, la tratamos como una heurística útil pero no necesariamente admisible ni consistente en todos los dominios posibles. En nuestras pruebas devolvió planes de la misma longitud que BFS, pero con menos expansiones.

## Punto 5: Planificación jerárquica de tareas – HTN

La parte jerárquica fue la que más se pareció a pensar la misión como la organizaría una persona. En vez de buscar desde cero entre todas las acciones primitivas, definimos tareas de alto nivel: *PrepareSupplies*, *ExtractPatient* y *Navigate*. La navegación se refina en movimientos sobre caminos mínimos; preparar suministros implica ir por ellos, llevarlos al puesto médico y ejecutar *SetupSupplies*; y extraer un paciente consiste en ir hasta su ubicación, recogerlo, volver al puesto médico y dejarlo allí. Sobre esa estructura, *FullRescueMission* compone las fases necesarias antes de ejecutar *Rescue*.

Cuando pasamos a múltiples pacientes apareció un error que fue muy útil para entender el dominio. Al comienzo, preparar suministros para cada paciente parecía razonable, pero en realidad hacía que la segunda misión intentara ejecutar *SetupSupplies* cuando el puesto médico ya tenía *SuppliesReady*. Eso chocaba directamente con la precondition negativa de la acción. La corrección fue preparar suministros una sola vez y luego rescatar los pacientes usando misiones que ya parten de un puesto médico abastecido.



## Puntos 2c, 3d, 4b y 5b: Resultados experimentales y comparación

Para no quedarnos solo con la impresión de que un algoritmo “parecía funcionar”, ejecutamos las pruebas con `python3 experiments.py`. En cada caso tomamos el plan producido, lo aplicamos desde el estado inicial y verificamos que el estado final cumpliera el objetivo. Los tiempos pueden variar un poco entre ejecuciones, pero la longitud del plan, los estados expandidos y la validez nos dieron una base más clara para comparar los enfoques.

| Layout      | Planificador           | Longitud | Expandidos | Tiempo (s) | Válido |
|-------------|------------------------|----------|------------|------------|--------|
| tinyBase    | forwardBFS             | 9        | 225        | 0.0086     | sí     |
| tinyBase    | backwardSearch         | 9        | 40         | 0.0013     | sí     |
| tinyBase    | A* ignorePreconditions | 9        | 252        | 0.0253     | sí     |
| tinyBase    | A* ignoreDeleteLists   | 9        | 25         | 0.0471     | sí     |
| smallRescue | forwardBFS             | 23       | 1181       | 0.1898     | sí     |
| smallRescue | backwardSearch         | 23       | 240        | 0.0050     | sí     |
| smallRescue | A* ignorePreconditions | 23       | 1316       | 0.2001     | sí     |
| smallRescue | A* ignoreDeleteLists   | 23       | 282        | 12.3826    | sí     |
| htnBase     | forwardBFS             | 12       | 830        | 0.1275     | sí     |
| htnBase     | backwardSearch         | 12       | 70         | 0.0011     | sí     |
| htnBase     | A* ignorePreconditions | 12       | 937        | 0.1383     | sí     |
| htnBase     | A* ignoreDeleteLists   | 12       | 64         | 2.2908     | sí     |

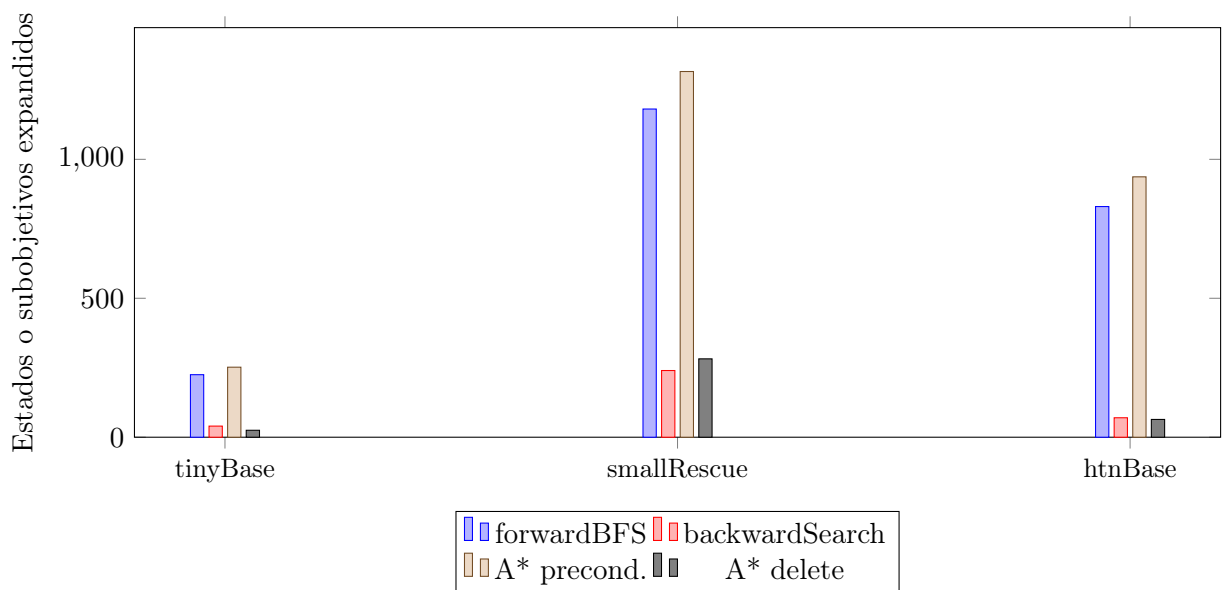


Figura 1: Comparación de estados expandidos en los layouts de rescate simple.

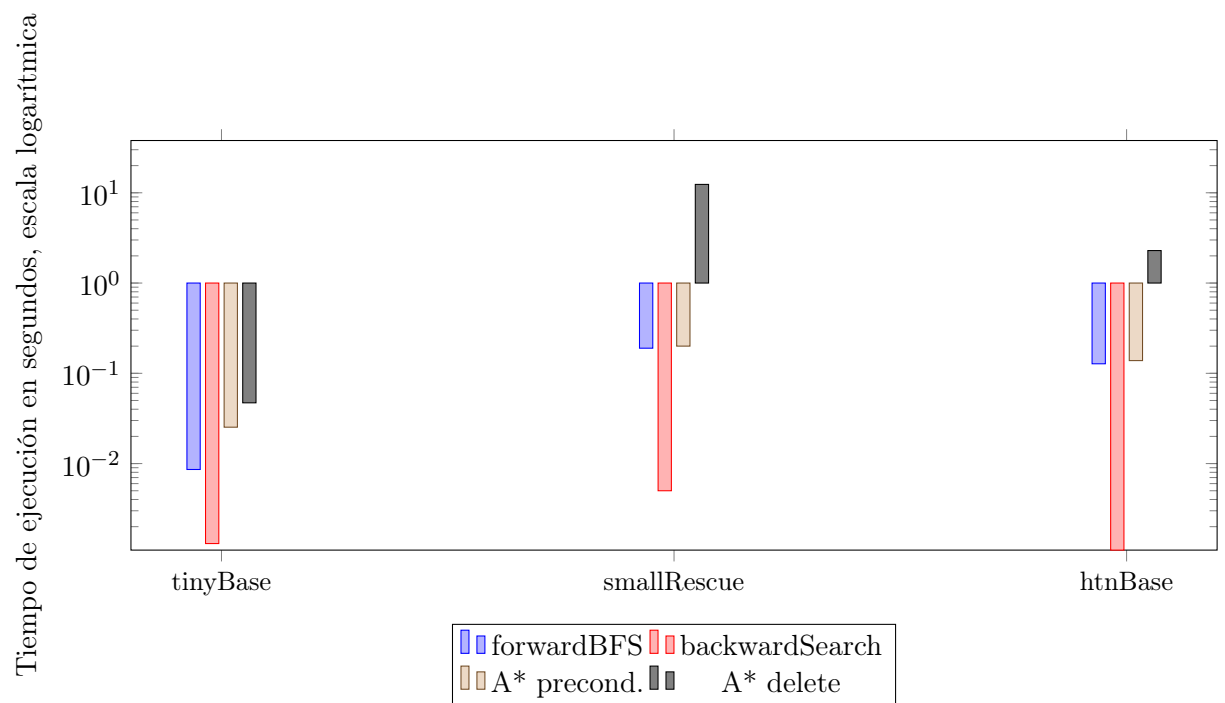


Figura 2: Comparación de tiempos. La escala logarítmica permite observar simultáneamente tiempos muy pequeños y el costo de calcular la heurística de delete lists.

Los resultados nos mostraron que *forwardBFS* es una buena referencia, pero no necesariamente el método más cómodo cuando el espacio empieza a crecer. La heurística de precondiciones ignoradas fue barata y admisible, aunque en la práctica guió poco: en los tres layouts terminó expandiendo incluso más que BFS. En cambio, ignorar delete lists sí dio una señal más útil sobre lo que faltaba

para completar la misión, reduciendo las expansiones de 1181 a 282 en *smallRescue* y de 830 a 64 en *htnBase*. La desventaja fue clara también: calcular esa heurística cuesta más tiempo.

La búsqueda regresiva encontró planes de la misma longitud que BFS y, con las podas de invariantes, exploró menos nodos en los layouts evaluados. Esto nos ayudó a entender que regresar desde la meta puede ser muy potente cuando el objetivo está bien definido, pero también que esa ventaja depende de controlar los subobjetivos imposibles que van apareciendo durante la regresión.

| Layout     | Orden HTN        | Longitud | Expandidos | Tiempo (s) | Válido |
|------------|------------------|----------|------------|------------|--------|
| tinyMulti  | orden del layout | 21       | 15         | 0.0002     | sí     |
| tinyMulti  | orden inverso    | 21       | 75         | 0.0004     | sí     |
| duoRescue  | orden del layout | 20       | 34         | 0.0003     | sí     |
| duoRescue  | orden inverso    | 20       | 130        | 0.0006     | sí     |
| smallMulti | orden del layout | 41       | 905        | 0.0076     | sí     |
| smallMulti | orden inverso    | 41       | 15205      | 0.0875     | sí     |



Figura 3: Impacto del orden de procesamiento de pacientes en la búsqueda HTN. La escala logarítmica evita que el caso *smallMulti* oculte las diferencias en los layouts pequeños.

En los layouts con varios pacientes, cambiar el orden de procesamiento no cambió la longitud final del plan, pero sí cambió mucho el esfuerzo de búsqueda. En *smallMulti*, el orden inverso mantuvo la longitud en 41, pero aumentó las expansiones de 905 a 15205. Esto nos pareció importante porque muestra que, incluso cuando el plan final tiene el mismo tamaño, la forma de refinar las tareas puede hacer que el planificador trabaje bastante más. En mapas más asimétricos, ese orden probablemente también afectaría la longitud del plan.

## Punto 6a: Análisis de complejidad, completitud y optimalidad

Para comparar los algoritmos usamos  $n$  como el número de fuentes que pueden aparecer en el dominio,  $a$  como el número de acciones aterrizadas disponibles,  $d$  como la profundidad del plan solución y  $b$  como el factor de ramificación efectivo. En este taller  $b$  queda acotado por  $a$ , pero suele ser mucho menor porque solo una parte de las acciones aterrizadas es aplicable en cada estado. En el peor caso el número de estados completos posibles puede crecer como  $O(2^n)$ , porque cada fuente puede estar presente o ausente, aunque los invariantes físicos del dominio reducen ese espacio en la práctica.

| Algoritmo      | Tiempo peor caso                                                                                                               | Espacio peor caso                                                            | Completitud y optimalidad                                                                                                                                                                                     |
|----------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| forwardBFS     | $O(a \cdot \min(2^n, b^d))$ , porque cada expansión puede revisar acciones aterrizadas para generar sucesores.                 | $O(\min(2^n, b^d))$ , por frontera y visitados.                              | Es completo en espacios finitos y óptimo si todas las acciones tienen el mismo costo. En el dominio del taller esto se cumple porque cada acción cuesta 1.                                                    |
| backwardSearch | $O(a_r \cdot \min(2^n, b_r^d))$ , donde $a_r$ son acciones relevantes para regresión y $b_r$ el ramificado sobre subobjetivos. | $O(\min(2^n, b_r^d))$ , por frontera regresiva y subobjetivos visitados.     | Es completo y óptimo bajo BFS si no se eliminan acciones necesarias. Las podas usadas corresponden a invariantes físicos y estáticos del dominio.                                                             |
| A*             | $O(a \cdot \min(2^n, b^d))$ en el peor caso, aunque una heurística informativa reduce expansiones en la práctica.              | $O(\min(2^n, b^d))$ , porque mantiene frontera prioritaria y mejores costos. | Es completo con costos positivos y espacio finito. Es óptimo si la heurística es admisible y consistente.                                                                                                     |
| HTN            | $O(r^k \cdot m)$ , con $r$ refinamientos por tarea, $k$ tareas abstractas por expandir y $m$ acciones primitivas validadas.    | $O(r^k \cdot m)$ en la frontera de planes jerárquicos.                       | Es completo respecto a los refinamientos definidos, no respecto a todos los planes clásicos posibles. Es óptimo solo si los refinamientos preservan todas las alternativas relevantes y se ordenan por costo. |

En *forwardBFS* el costo aparece porque el algoritmo explora desde el inicio sin saber qué acciones son más prometedoras para llegar al objetivo. Eso lo vuelve confiable y fácil de revisar, pero también hace que crezca rápido. En *smallRescue* expandió 1181 estados para encontrar un plan de 23 acciones. El resultado confirma la completitud y optimalidad esperadas, pero también muestra por qué BFS se vuelve pesado cuando el mapa o el número de objetos aumenta.

En *backwardSearch*, la pregunta cambia: en vez de explorar todo lo alcanzable desde el inicio, el algoritmo pregunta qué tendría que haber sido cierto antes para lograr *Rescued(p)*. Esa dirección redujo la búsqueda en nuestros layouts. En *smallRescue* expandió 240 subobjetivos, bastante menos que *forwardBFS*. Al mismo tiempo, vimos que la regresión necesita cuidado, porque puede inventar

metas parciales inconsistentes si no se revisan invariantes como ubicación única, predicados estáticos y capacidad de carga del robot.

Con A\* entendimos que no basta con tener una heurística correcta; también tiene que decir algo útil sobre el problema. *ignorePreconditionsHeuristic* es admisible en este dominio porque cuenta flujos de rescate pendientes y cada uno exige al menos una acción *Rescue*. También se comportó de manera consistente en los layouts evaluados. Aun así, fue poco informativa porque no distingue si el robot está cerca de los suministros, del paciente o del puesto médico; solo ve cuántos objetivos de rescate faltan.

La heurística *ignoreDeleteListsHeuristic* fue más informativa para este dominio porque conserva precondiciones y acumula efectos positivos, lo que se parece más a la cadena real de preparar suministros, mover pacientes y rescatar. La evidencia fue clara: en *htnBase* redujo las expansiones de 830 a 64, y en *smallRescue* de 1181 a 282. La otra cara fue el tiempo: resolver muchas veces el problema relajado con hill-climbing hizo que algunos casos tardaran más. Por eso no la presentamos como una heurística universalmente mejor, sino como una guía más fuerte pero más costosa.

HTN nos dio una ventaja distinta. No solo ordena mejor la búsqueda, sino que cambia el espacio sobre el que se busca. En lugar de probar combinaciones arbitrarias de acciones primitivas, parte de una estructura de misión: preparar suministros, extraer paciente y rescatar. Cuando esa estructura es correcta, la complejidad efectiva baja mucho porque el planificador ya no tiene que redescubrir desde cero una secuencia que en el dominio tiene sentido operativo. El experimento con orden inverso también mostró que el orden de refinamiento puede multiplicar el esfuerzo aunque la longitud final no cambie.

## Punto 6b: Reflexión sobre el trabajo y el uso de IA

Después de trabajar en la implementación, sentimos que el mayor reto no fue escribir operaciones de conjuntos, sino lograr que esas operaciones representaran bien el mundo del problema. Una precondición negativa como *not SuppliesReady* parece un detalle pequeño, pero fue suficiente para romper la planificación HTN multi-paciente cuando intentábamos preparar suministros varias veces. Ese error nos ayudó a entender que un planificador no falla solamente por el algoritmo de búsqueda; también falla cuando el modelo del dominio no captura bien sus restricciones.

El apoyo de IA nos sirvió en varias etapas, no solamente como revisión final. En las primeras iteraciones nos ayudó a notar que algunas partes del repositorio funcionaban en layouts pequeños, pero todavía no cumplían bien la intención algorítmica del taller. Esa diferencia fue importante: no es lo mismo obtener un plan válido una vez que implementar correctamente regresión BFS, A\* o HTN. A partir de esa revisión, las correcciones dejaron de ser ajustes superficiales y se conectaron con propiedades como completitud, optimalidad, admisibilidad y consistencia.

Lo más formativo fue tener que entender las correcciones antes de aceptarlas. Por ejemplo, en regresión vimos que una implementación puede usar bien la fórmula  $(g - \text{ADD}(a)) \cup \text{PRECOND}(a)$  y aun así generar subobjetivos imposibles si no se incorporan invariantes del dominio. En HTN, la IA ayudó a señalar que preparar suministros una vez por paciente era incompatible con la precondición negativa de *SetupSupplies*; pero la explicación que finalmente nos quedó fue nuestra: en una misión de rescate, un puesto médico abastecido puede atender varios pacientes sin repetir la preparación.

Construir el prompt adecuado tampoco fue inmediato. Las primeras preguntas, más generales, produ-



cían respuestas útiles pero todavía muy amplias. Para avanzar de verdad tuvimos que pedir evidencia: contrastar código con la rúbrica, ejecutar pruebas, medir expansiones y explicar los resultados usando los conceptos del curso. Esa iteración nos dejó una lección práctica: la IA ayuda más cuando se le pide justificar y verificar, no solo opinar si algo “está bien”.

Consideramos que la solución final sí podríamos explicarla en una sustentación. No porque sepamos cada línea de memoria, sino porque entendemos las decisiones principales: BFS sirve como referencia óptima con costos unitarios, la regresión razona desde las metas, A\* depende fuertemente de la heurística y HTN reduce el espacio al usar conocimiento de alto nivel sobre la misión. La principal limitación del uso de IA fue que algunas sugerencias podían sonar convincentes sin ser suficientes por sí solas. Por eso fue indispensable ejecutar el código, revisar planes inválidos, comparar métricas y escribir nuestra propia interpretación.

## Apéndice: Trazabilidad del uso de IA

Durante el taller usamos IA principalmente para revisar, depurar, contrastar contra el enunciado e interpretar resultados, no como reemplazo de la implementación inicial. La interacción fue más útil cuando pedimos revisiones concretas sobre partes del planificador y luego convertimos esas respuestas en cambios que pudiéramos explicar con los conceptos de planificación vistos en clase. Como trazabilidad, estos son ejemplos representativos de los prompts usados durante la co-construcción de la solución:

| Momento de uso                      | Ejemplo de prompt                                                                                                               |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Revisión de búsqueda hacia adelante | “Revisa si esta implementación de <i>forwardBFS</i> cumple con la rúbrica del taller y dime qué casos faltan validar.”          |
| Revisión de regresión               | “Compara <i>backwardSearch</i> contra la fórmula de regresión del enunciado y revisa si las podas conservan corrección.”        |
| Análisis de heurísticas             | “Ayúdame a interpretar esta tabla de resultados de BFS, <i>backwardSearch</i> y A* sin exagerar las conclusiones.”              |
| Revisión de HTN multi-paciente      | “Verifica si el refinamiento HTN para múltiples pacientes respeta las precondiciones de <i>SetupSupplies</i> y <i>Rescue</i> .” |
| Redacción del informe               | “Revisa la reflexión final para que explique qué aprendimos del uso de IA y no suene genérica.”                                 |

La versión activa del proyecto corresponde a la implementación final validada con `experiments.py`. Los cambios relevantes quedaron reflejados en los archivos modificados del repositorio y en los resultados experimentales reportados en este documento. Esta trazabilidad fue importante porque nos obligó a separar la ayuda recibida de nuestra comprensión propia: cada sugerencia se aceptó solo después de contrastarla con el enunciado, ejecutar pruebas y verificar que los planes generados alcanzaran los objetivos de *SimpleRescueProblem* y *MultiRescueProblem*.

## Conclusión: Aprendizajes para sistemas de respuesta a desastres

Después de implementar y probar los algoritmos, nos quedó claro que no basta con que el planificador encuentre una secuencia válida una vez. En varios momentos el código funcionaba en un layout

pequeño, pero fallaba cuando cambiaban las condiciones del mapa o aparecían varios pacientes. Por eso terminamos dándole más importancia a validar los planes, revisar las precondiciones y comparar métricas que a una sola ejecución exitosa.

También vimos que cada enfoque sirve para algo distinto. BFS fue útil como referencia porque es fácil de verificar y mantiene optimalidad con costos unitarios, pero crece rápido. La regresión fue más eficiente en nuestros layouts porque parte de la meta y evita recorrer muchos estados que no aportan al objetivo. A\* dependió mucho de la heurística: ignorar precondiciones fue barato pero débil, mientras que ignorar delete lists redujo expansiones a costa de más tiempo. HTN fue el enfoque que más se pareció a cómo uno organizaría manualmente una misión: preparar suministros, extraer paciente y rescatar.

Para futuros sistemas de respuesta a desastres, nos parece que lo más valioso sería combinar estas ideas en lugar de depender de un solo algoritmo. La planificación clásica aporta garantías y una base clara para validar soluciones; las heurísticas ayudan a orientar la búsqueda cuando el espacio crece; y HTN permite incorporar protocolos operativos parecidos a los que seguiría un equipo humano. Esa combinación puede producir planes más claros, más fáciles de revisar y más útiles en escenarios donde el tiempo de respuesta importa tanto como encontrar una solución.